NORTHERN UNIVERSITY
OF BUSINESS & TECHNOLOGY KHULNA
Knowledge for an Enlightening Future

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

System Design Document

ON

AI Programming Tutor & Bug Explainer

Course Code: CSE 4104

Course Title: Software Development III

SUBMITTED TO

Name: Md. Riaz Mahmud

Assistant Professor

Department of CSE

Northern University of Business & Technology, Khulna.

SUBMITTED BY

Team Name	CSE4104 - 7B - T04
Section	7B
Project Title	AI Programming Tutor & Bug Explainer
Member Names	M.M. Asfy Or Rayhan Alisha Rahman Nadia Afrin Fatima Tanjim Rafa
Student IDs	11230121106 11230121112 11230121121 11230321556
GitHub Repository	https://github.com/Rayhan-55/cse4104-7b-t04-ai-programming-tutor-and-Bug-Explainer

Date of Submission: 15/06/2026

Team Information

Team Name	CSE4104 - 7B - T04
Section	7B
Project Title	AI Programming Tutor & Bug Explainer
Team Leader	M.M. Asfy Or Rayhan
Member Names	M.M. Asfy Or Rayhan Alisha Rahman Nadia Afrin Fatima Tanjim Rafa
Student IDs	11230121106 11230121112 11230121121 11230321556
GitHub Repository	https://github.com/Rayhan-55/cse4104-7b-t04-ai-programming-tutor-and-Bug-Explainer

Contents

1. **Project Information** 1

2. **Architecture Diagram**.....8

3. **ER Diagram**.....9

4. **Use Case Diagram**.....10

5. **Activity Diagram**.....11

6. **API Design**.....12

7. **AI Workflow**.....17

8. **References**.....22

System Design Document

AI Programming Tutor & Bug Explainer

1. Project Information

1.1 Project Overview

AI Programming Tutor & Bug Explainer is a full-stack web application built to help beginner programmers understand code, fix their mistakes, and make sense of the error messages that usually discourage them early in their learning. Rather than acting as a search engine or a code generator, the system behaves like a patient tutor: it explains a concept in plain words, points out where a piece of code goes wrong, rewrites compiler and runtime errors in simple language, and suggests small, realistic improvements that a learner can actually apply. The application is divided into a React-based client and a Node.js/Express server. Every request that needs intelligence is forwarded from the client to the server, and the server alone communicates with the language-model provider. This design keeps the provider credentials private and allows the underlying model to be replaced without touching the user interface. A distinctive property of the implementation is that the platform remains usable even when no model key and no database are configured; in that situation it falls back to built-in sample responses and disables only the features that require persistence.

1.2 Objectives

The project was developed with the following objectives in mind:

1. To lower the entry barrier for first-time programmers by explaining concepts and code in approachable, bilingual (Bangla and English) language.
2. To translate intimidating compiler and runtime errors into clear explanations that state the meaning, the likely cause, and a concrete fix.
3. To centralise all model communication behind a single service layer so that the provider can be switched through configuration alone.

4. To support guests immediately while offering registered learners persistent history, progress tracking, and a personal memory that improves future answers.
5. To keep the system dependable through graceful fallback, so that a missing credential or a provider failure never results in a broken request.

1.3 Core Features

The feature set was derived directly from the implemented controllers, routes, and client pages. The principal features are summarised below.

Feature	Description
AI Tutor	Explains any programming concept using a short definition, a real-world analogy, a small code example, and a line of encouragement.
Bug Explainer	Examines pasted code, describes the defect line by line, provides a corrected version, and advises how to avoid the mistake.
Compiler Error Translator	Converts raw error text into a plain-language explanation covering its meaning, common cause, and a corrective step.
Code Improver	Reviews working code and recommends clearer naming, cleaner syntax, and reductions in time complexity.
GitHub Repository Explainer	Reads a public repository, produces a project overview, and explains individual files on demand.
Document & Image Explainer	Extracts and explains the contents of uploaded PDF notes, or interprets a photograph of handwritten code through the vision model.
Practice Problem Generator	Produces a beginner-level problem in structured JSON form, including statement, sample input/output, hints, and a solution.

Feature	Description
Memory & Personalisation	Records short notes about a learner's activity and weak topics, and feeds a summary back into later requests.
Voice Tutor	Reads any generated explanation aloud using the browser speech engine.
Gamified Dashboard	Tracks experience points, level, streak, and recent activity for registered users.
Administration Panel	Provides platform statistics, tool-usage figures, a user list with role management, and a recent-activity feed.

1.4 Target Users

Three groups of users were considered during the design of the system:

- **Beginning students** of programming, particularly those who are more comfortable when explanations mix Bangla and English.
- **Self-taught learners** who frequently encounter compiler errors and need them rephrased in understandable terms.
- **Instructors and administrators** who require a lightweight overview of how the learning tools are being used across the platform.

1.5 Technology Stack

The technologies listed below were identified from the project's dependency files and source modules.

Layer	Technologies	Role in the System
Frontend	React 18, Vite	Single-page interface and build tooling.
Styling & Animation	Tailwind CSS, Framer Motion	Utility-first styling and interface animation.
Editor & Charts	Monaco Editor, Recharts	In-browser code editing and dashboard charts.
Client Networking	Axios	Sends API requests and attaches the authentication token.
Backend	Node.js, Express 4	REST API, routing, and middleware.
Database	MongoDB, Mongoose	Stores users, history, weak topics, and memory (optional).
Authentication	JSON Web Token, bcryptjs	Stateless sessions and password hashing.
File Handling	Multer, unpdf	In-memory uploads and PDF text extraction.
Request Control	express-rate-limit	Throttles the AI-related endpoints.
Model Providers	Groq, Google Gemini, OpenAI, Mock	Interchangeable text and vision back-ends.

1.6 Data Models / Database Design

Persistence is provided through Mongoose models and is intentionally optional. When the database connection string is absent, the API detects that the database is not ready and skips all write operations without raising an error. Four collections are defined.

1.6.1 User

Field	Type	Description
name	String	Required display name.
email	String	Required, unique, stored in lowercase.
password	String	Required (minimum six characters), hashed with bcrypt, never returned in responses.
role	String	Either student or admin; defaults to student.
xp, level, streak	Number	Progress values; level is derived as floor(xp / 100) + 1.
lastActiveDate	Date	Used to maintain the activity streak.
subscription	String	Either free or pro; defaults to free.

1.6.2 ChatHistory

Field	Type	Description
userId	ObjectId	Reference to the owning user (indexed).
feature	String	One of tutor, bugExplainer, errorTranslator, codeImprover, problemGenerator.
message	String	The input submitted by the user.
response	String	The answer returned by the model.

Field	Type	Description
language	String	Programming or interface language.
topic	String	Optional grouping label.

1.6.3 WeakTopic

Field	Type	Description
userId	ObjectId	Reference to the owning user.
topic	String	A topic the learner struggles with, such as recursion or pointers.
mistakeCount	Number	Number of recorded difficulties with the topic.
confidenceLevel	Number	A score from 0 (weak) to 100 (strong).

A compound unique index on (userId, topic) guarantees a single record per learner per topic.

1.6.4 Memory

Field	Type	Description
userId	ObjectId	Reference to the owning user (unique).
notes	Array of { text, createdAt }	Short factual notes about the learner, capped to the forty most recent entries.

1.7 Environment Configuration

All secrets and provider selections are supplied through environment variables in the server. Defaults are chosen so that the application can start with no configuration at all.

Variable	Default	Purpose
AI_PROVIDER	mock	Selects the active provider: groq, gemini, openai, or mock.
GROQ_API_KEY	(empty)	Free-tier key; recommended for a zero-cost deployment.
GEMINI_API_KEY	(empty)	Free-tier key supporting both text and vision.
OPENAI_API_KEY	(empty)	Paid, OpenAI-compatible key.
MONGO_URI	(empty)	Enables accounts, history, memory, and administration.
JWT_SECRET	(required)	Secret used to sign authentication tokens.
JWT_EXPIRES_IN	7d	Lifetime of an issued token.
ADMIN_EMAILS	(empty)	Comma-separated list of emails granted the admin role.
GITHUB_TOKEN	(optional)	Raises the GitHub request limit from 60 to 5000 per hour.
CLIENT_URL	localhost:5173	Origin permitted by the CORS policy.

2. Architecture Diagram

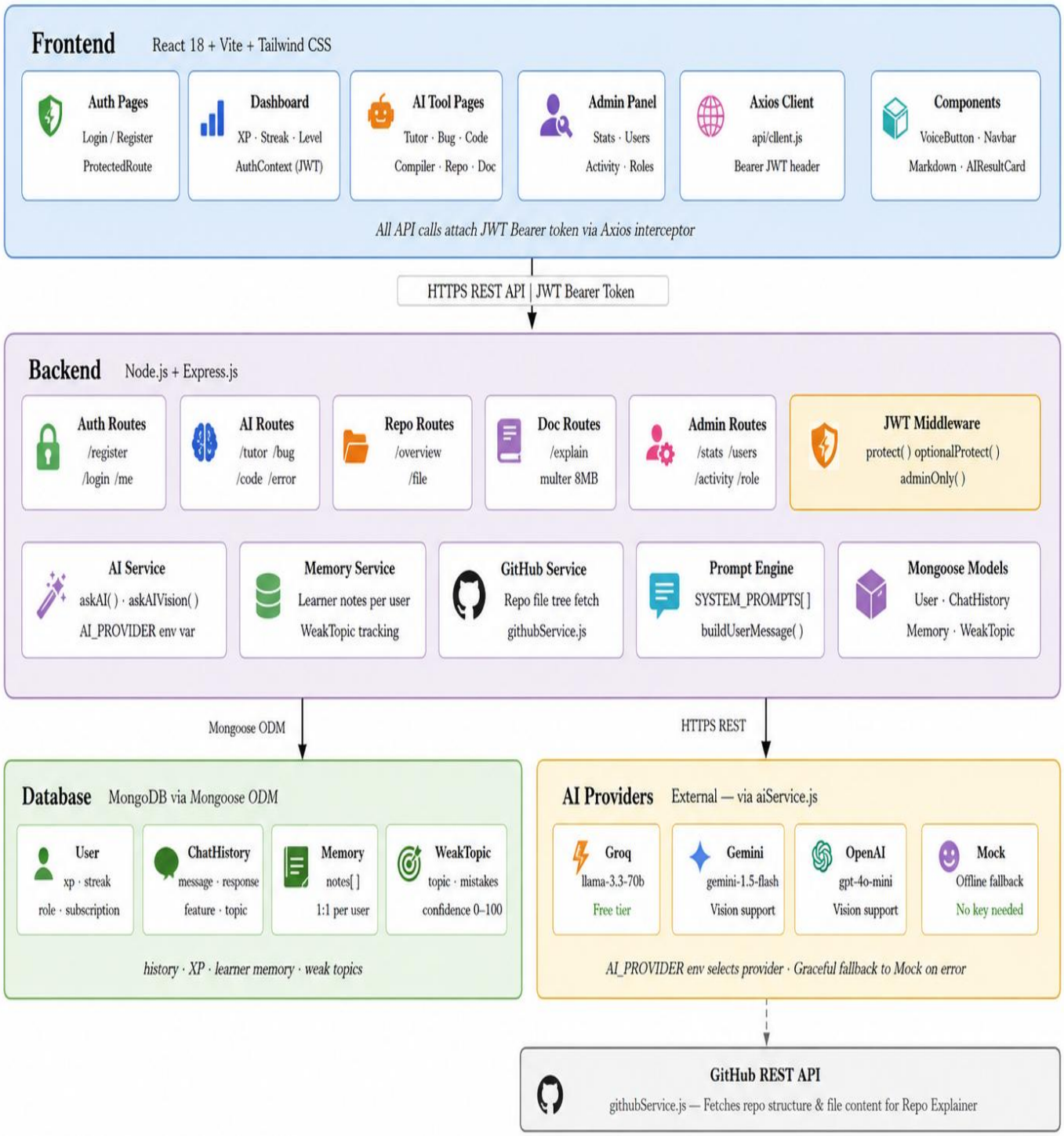


Figure: Architecture Diagram of the AI Programming Tutor and Bug Explainer

3. ER Diagram

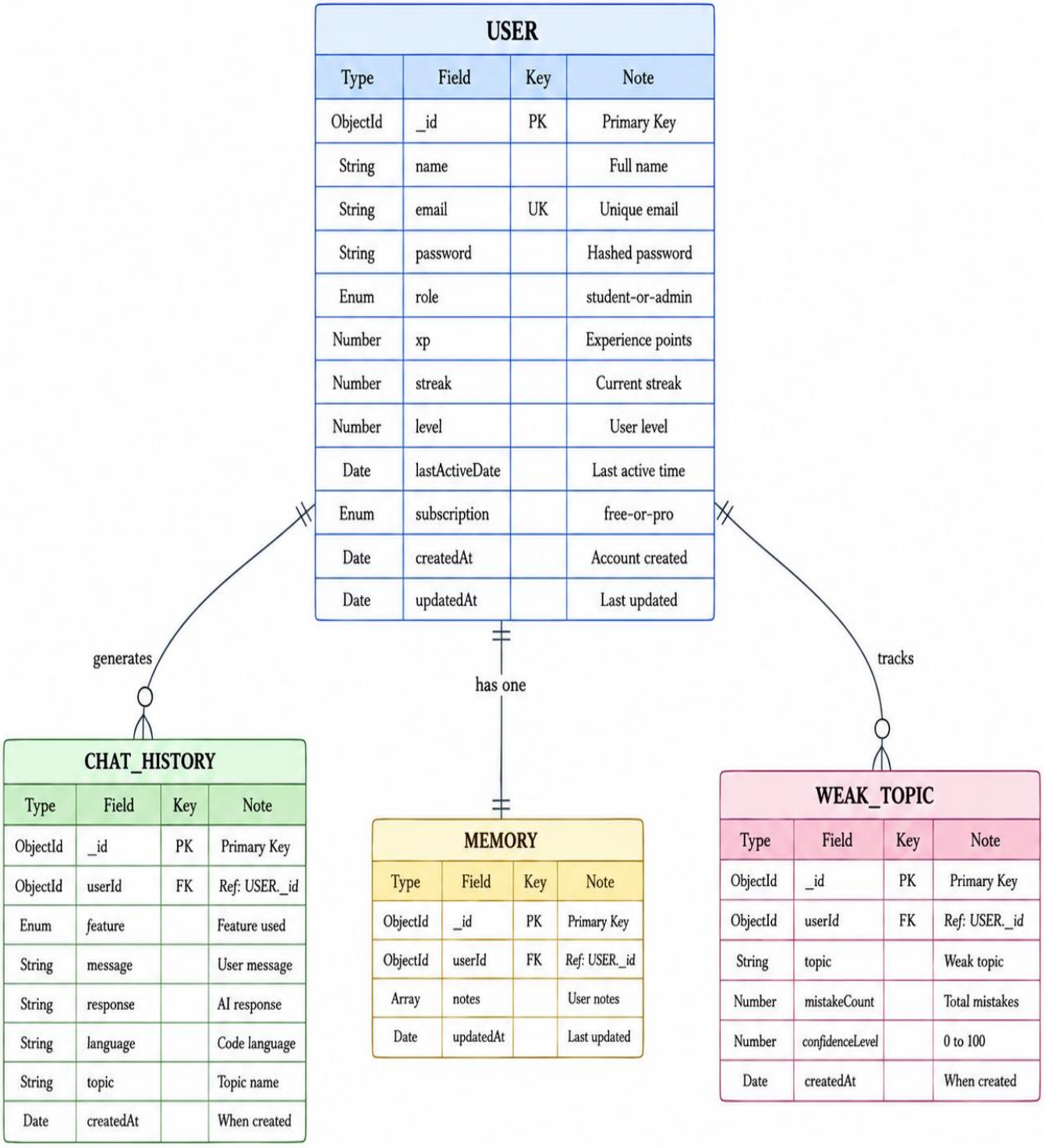


Figure: Entity Relationship (ER) Diagram of the AI Programming Tutor and Bug Explainer

4. Use Case Diagram

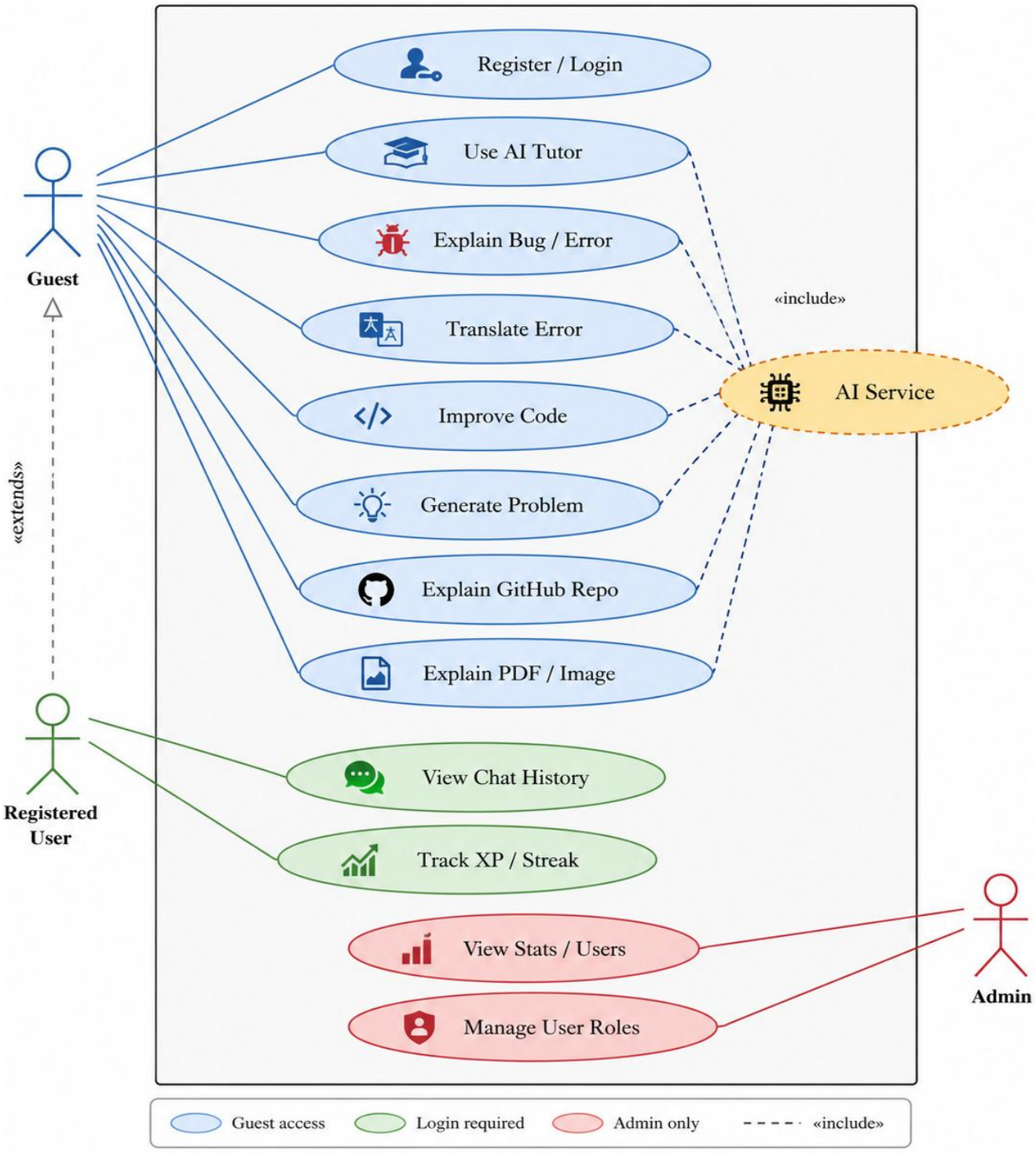


Figure: Use Case Diagram of the AI Programming Tutor and Bug Explainer System

5. Activity Diagram

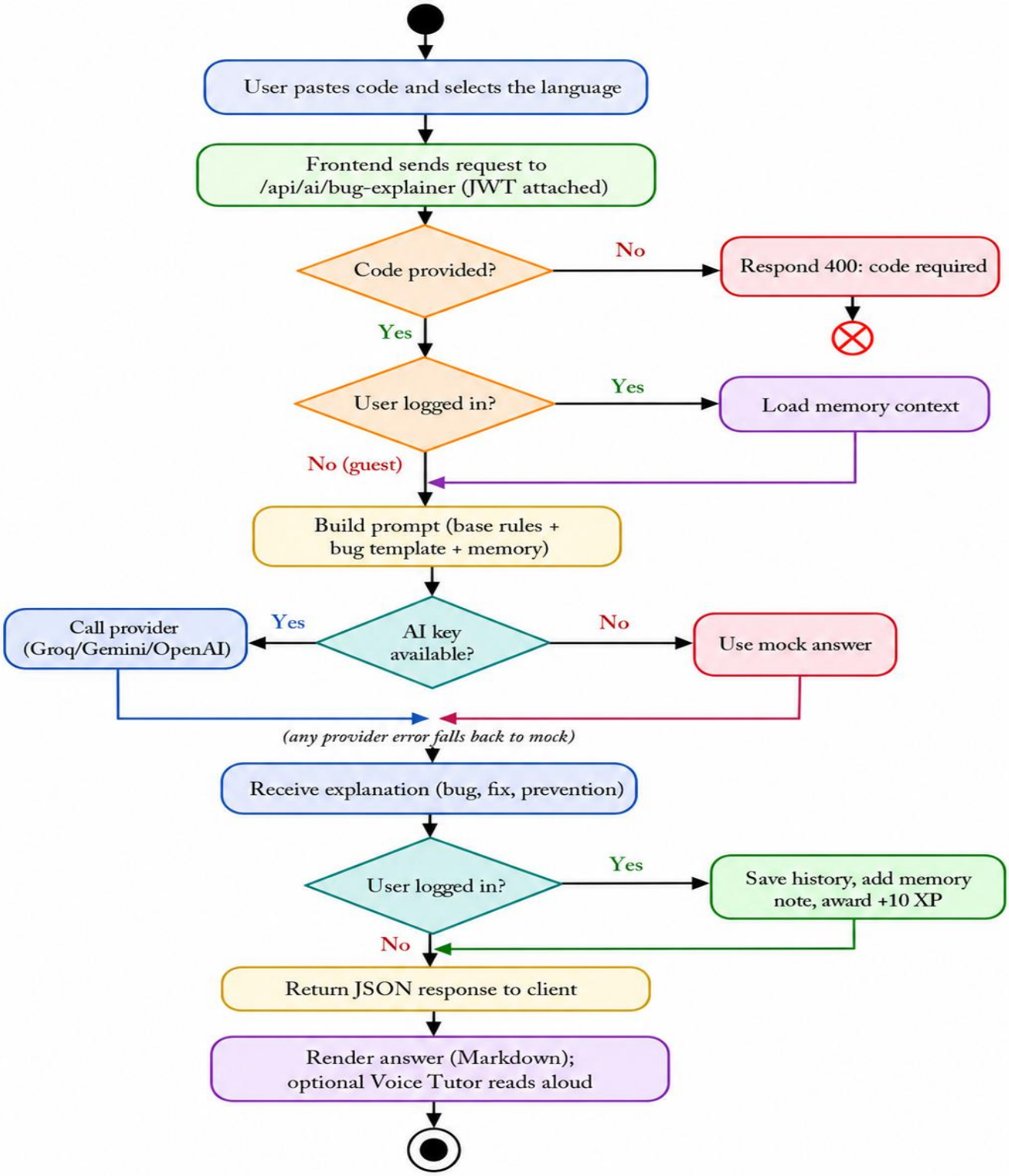


Figure: Activity Diagram of the AI Programming Tutor and Bug Explainer

6. API Design

The server exposes a REST interface under the base path /api. All request and response bodies are JSON, except file uploads, which use multipart form data. Authentication is stateless and relies on a JSON Web Token transmitted in the Authorization: Bearer <token> header. The endpoints that invoke the model are limited to thirty requests per minute per client. Four access levels are used throughout the interface. None indicates a public endpoint. Optional indicates an endpoint that works for guests but records history and awards experience when a valid token is present. Bearer indicates that authentication is mandatory. Admin indicates that the authenticated user must hold the administrator role.

6.1 Authentication APIs

Method	Endpoint	Purpose	Input Data	Expected Output	Auth
POST	/api/auth/register	Create a new account and return a signed token.	name, email, password	User object and JWT	None
POST	/api/auth/login	Authenticate an existing user.	email, password	User object and JWT	None
GET	/api/auth/me	Return the currently authenticated user.	(none)	User object	Bearer

The implementation does not expose a server-side logout endpoint. Because sessions are stateless, the client logs the user out by discarding the stored token, after which the protected endpoints reject further requests automatically.

6.2 User APIs

Method	Endpoint	Purpose	Input Data	Expected Output	Auth
GET	/api/ai/history	Retrieve the user's fifty most recent activities.	(none)	Array of history records	Bearer

6.3 AI APIs

Method	Endpoint	Purpose	Input Data	Expected Output	Auth
POST	/api/ai/tutor	Explain a programming concept.	topic, language	Explanation text	Optional
POST	/api/ai/bug-explainer	Find and explain a bug, with a fix.	code, language	Explanation text	Optional
POST	/api/ai/error-translator	Translate a compiler/runtime error.	error, language	Explanation text	Optional
POST	/api/ai/code-improver	Suggest improvements to working code.	code, language	Suggestions text	Optional
POST	/api/ai/problem	Generate a practice problem.	topic, difficulty	Problem as JSON	Optional

Method	Endpoint	Purpose	Input Data	Expected Output	Auth
POST	/api/repo/overview	Explain a public GitHub repository.	url, lang	Metadata, file tree, explanation	Optional
POST	/api/repo/file	Explain a single file in a repository.	url, path, lang	File explanation	Optional
POST	/api/doc/explain	Explain an uploaded PDF or image.	file, lang, question	Explanation text	Optional

6.4 Admin APIs

Method	Endpoint	Purpose	Input Data	Expected Output	Auth
GET	/api/admin/stats	Return platform statistics and tool usage.	(none)	Statistics object	Admin
GET	/api/admin/users	List platform users (up to 200).	(none)	Array of users	Admin
GET	/api/admin/activity	Return recent activity across users.	(none)	Activity records	Admin

Method	Endpoint	Purpose	Input Data	Expected Output	Auth
PATCH	/api/admin/users/:id/role	Promote or demote a user.	role	Updated user	Admin

6.5 Other APIs

Method	Endpoint	Purpose	Input Data	Expected Output	Auth
GET	/api/health	Report service status and active provider.	(none)	Status object	None

6.6 Error Responses and Status Codes

Errors are produced by a central handler and follow a single shape, { "message": "reason" }, with a stack trace added only in the development environment. The status codes used by the interface are listed below.

Status	Meaning within the System
200 / 201	Successful request / resource created.
400	A required field is missing or invalid.
401	The token is missing, expired, or invalid.
403	The user is authenticated but is not an administrator.

Status	Meaning within the System
404	The route or repository could not be found.
409	The email address is already registered.
413 / 415	The uploaded file is too large / of an unsupported type.
422	The uploaded PDF could not be read or contained no selectable text.
429	The per-minute request limit was exceeded.
503	A database feature was requested but no connection is configured.

7. AI Workflow

7.1 The Need for AI in the Project

The purpose of the platform cannot be met by conventional programming alone. Explaining an arbitrary concept, reasoning about why a particular piece of code fails, and rephrasing an unfamiliar error message all require natural-language understanding and generation. A large language model supplies this capability. It allows the system to respond to inputs that were never explicitly anticipated, to adapt its tone to a beginner audience, and to produce explanations in a mixture of Bangla and English that suits the target users.

7.2 Model Services Used

The system is provider-agnostic. The active provider is chosen through a single environment variable, and the service layer maps each provider to an appropriate text model and, where supported, a vision model. The Groq and OpenAI providers share one OpenAI-compatible code path; Gemini uses its own client; and a mock provider produces fixed sample answers when no key is available.

Provider	Default Text Model	Default Vision Model
Groq	llama-3.3-70b-versatile	meta-llama/llama-4-scout-17b-16e-instruct
Google Gemini	gemini-1.5-flash	gemini-1.5-flash
OpenAI	gpt-4o-mini	gpt-4o-mini
Mock	built-in sample responses	built-in sample responses

7.3 How the Integration Works

All intelligence is concentrated in one service module that exposes two functions: askAI() for text features and askAIVision() for the image feature. Each function looks up the system prompt for the requested feature, assembles the user-facing message, and routes the call to the configured provider. No controller ever contacts a provider directly, which means the entire integration can be reconfigured or replaced by editing this one module.

7.4 Input Provided to the Model

The input sent to the model is constructed in two parts. The first is a system prompt that establishes the tutor's behaviour for the selected feature. The second is the user message, built from the data submitted to the endpoint — for example the concept to explain, the code to inspect, or the error text to translate — optionally preceded by a short personalisation summary for signed-in learners. A language directive is appended so that the answer is produced in Bangla, English, or a Latin-script mixture of the two.

7.5 Output Generated by the Model

The output is a structured natural-language explanation whose shape is dictated by the feature's prompt. The tutor returns a definition, an analogy, an example, and encouragement; the bug explainer returns an observation, a line-by-line cause, a corrected snippet, and prevention advice; the problem generator is constrained to return valid JSON. The response is delivered to the client, rendered as formatted text, and may be read aloud by the voice component.

7.6 Contribution of AI to the Project

The model elevates the platform from a static reference into an interactive tutor. It allows a single interface to serve an unbounded range of questions, produces beginner-appropriate explanations on demand, and, in combination with the memory subsystem, tailors its answers to an individual learner's history. These qualities are difficult to reproduce with rule-based logic and represent the central value of the system.

7.7 AI Architecture and Request Flow

The end-to-end path of an intelligent request is summarised below and illustrated in the activity diagram that follows.

```
User Request
  |
Frontend (React) -- attaches JWT, sends JSON
  |
Backend API (Express route + middleware + controller)
  |
AI Service Layer (askAI / askAIVision: prompt + message + provider routing)
  |
Model Provider (Groq / Gemini / OpenAI)   or   Mock fallback
  |
Generated Response
  |
Frontend Display (formatted text, optional voice output)
```

Each stage performs a specific responsibility. The frontend collects the user's input and attaches the authentication token. The route applies rate limiting and authentication middleware. The controller validates the input and, for signed-in users, requests personalisation context. The service layer combines the prompt, the message, and the context, then dispatches the call to the active provider. The provider's reply — or, on any failure, a mock reply — is returned to the controller, which records the interaction before sending the answer back for display.

7.8 Prompt Handling Strategy

All prompts share a common base that fixes the tutor's personality: explanations are kept simple, jargon is avoided, the tone stays encouraging, Bangla and English are mixed to match the learner, and answers are kept short with small analogies and examples. Each feature then adds a task-specific template that fixes the structure of the reply, as shown below.

Feature	Required Output Structure
tutor	Definition, analogy, small example, and a closing line of encouragement.
bugExplainer	Observation, line-by-line cause, corrected code, and prevention advice.
errorTranslator	Plain meaning, reason it occurs, corrective step, and a one-line example.
codeImprover	Strengths, suggestions on naming, syntax and complexity, and an improved snippet.
repoOverview	One-line summary, purpose, technologies, key files, and how to run.
fileExplain	The file's role, a step-by-step walkthrough, and clarification of difficult terms.
problemGenerator	Valid JSON containing title, statement, sample input/output, hints, and solution.

7.9 Error Handling and Fallback Mechanism

Reliability is treated as a first-class concern. If the configured provider has no key, or if a provider call raises an error, the service layer silently substitutes a built-in mock answer so that the user still

receives a useful response. Database writes — history, memory, and experience updates — are performed on a best-effort basis and are skipped without error when no database is connected. Upload size limits, body-size limits, and input truncation of roughly sixteen thousand characters guard the model's token budget, and the per-minute rate limit protects the provider quota.

7.10 Personalisation and Memory Workflow

For authenticated users the platform maintains a lightweight memory that makes successive answers feel continuous. After each interaction a short note is appended to the user's memory record, which is capped at the forty most recent entries. Before a personalised answer is generated, a context builder assembles a compact summary from the latest notes together with the learner's most frequent weak topics, and this summary is prepended to the user message.

```
getContext(userId):
    recent = last 6 memory notes
    weak   = top 5 weak topics by mistakeCount
    return  a short summary of recent work and recurring difficulties


The summary is inserted ahead of the user message so the model can refer to it.
```

This subsystem is entirely optional. When no database is present the context builder returns an empty string, and the tutor continues to function for every user without personalisation.

8. References

8.1 Frameworks and Libraries

- React — https://react.dev
- Vite — https://vitejs.dev
- Tailwind CSS — https://tailwindcss.com/docs
- Framer Motion — https://www.framer.com/motion
- Monaco Editor — https://github.com/suren-atoyan/monaco-react
- Recharts — https://recharts.org
- Axios — https://axios-http.com/docs/intro
- Node.js — https://nodejs.org/en/docs
- Express — https://expressjs.com
- Mongoose — https://mongoosejs.com/docs
- jsonwebtoken — https://github.com/auth0/node-jsonwebtoken
- bcryptjs — https://github.com/dcodeIO/bcrypt.js
- Multer — https://github.com/expressjs/multer
- unpdf — https://github.com/unjs/unpdf
- express-rate-limit — https://github.com/express-rate-limit/express-rate-limit

8.2 AI Services Used

- Groq — https://console.groq.com/docs

- Google Gemini API — https://ai.google.dev/gemini-api/docs
- OpenAI API — https://platform.openai.com/docs

8.3 External APIs

- GitHub REST API — https://docs.github.com/en/rest

8.4 Database Technologies

- MongoDB — https://www.mongodb.com/docs
- MongoDB Atlas — https://www.mongodb.com/docs/atlas

8.5 Hosting Platforms

- Render — https://render.com/docs
- Railway — https://docs.railway.app
- Vercel — https://vercel.com/docs

8.6 Standards and Concepts Used

- REST (Representational State Transfer) as the architectural style for the API
- JWT (JSON Web Token) for stateless authentication — https://jwt.io/introduction
- HTTP status code semantics as defined by MDN Web Docs — https://developer.mozilla.org/en-US/docs/Web/HTTP/Status
- Model–View separation between the React client and the Express service layer
- The project's own source code (server and client directories) as the primary reference for this document